

utfcode
utf8.sty 3.10 UTF-8 input encoding 13.06.2000
scanner for code UTF-8 installed.

sentinel-rt: A Dynamic C++/CUDA Memory Safety Auditor and Structure-Aware Fuzzer for Native AI Runtimes

Kamran Saberifard

Department of Computer Engineering (Cybersecurity), MehrAstAn University, Guilan, Iran
Aryorithm Group — Global AI Infrastructure & Security Research
kamisaberi@gmail.com — ORCID: [0009-0002-7822-6168](https://orcid.org/0009-0002-7822-6168)

August 11, 2026

Abstract

Modern deep learning execution runtimes (e.g., `vLLM`, `llama.cpp`, `LibTorch`, `ONNX Runtime`) increasingly rely on compiled C++20 and custom CUDA backends to maximize hardware FLOPS and achieve strict sub-50ms inference Service Level Agreements (SLAs). However, these low-level execution engines prioritize raw computational speed over memory safety, exposing enterprise AI infrastructure to severe low-level memory corruption vulnerabilities. Integer overflows during model header deserialization, heap-based buffer overflows, Use-After-Free (UAF) conditions in custom tensor caching allocators, and GPU VRAM boundary violations can be exploited to achieve Remote Code Execution (RCE) or exfiltrate private tensor data across multi-tenant GPU nodes. In this paper, we present **sentinel-rt**, an open-source, high-throughput memory auditing and structure-aware fuzzing framework built specifically for native AI execution runtimes. **sentinel-rt** combines zero-copy C++20 format parsers for GGUF, Safetensors, and ONNX models with custom AFL++ grammar mutators, dynamic CUDA VRAM redzone allocators, and automated POSIX/ASan crash triaging. Our empirical evaluation demonstrates that **sentinel-rt** audits model deserialization and KV-cache allocations with less than 3.2% performance overhead while effectively discovering out-of-bounds reads and memory boundary breaches in native AI execution backends.

Keywords: AI Security, C++20, CUDA Memory Safety, AFL++, LibFuzzer, AddressSanitizer, GGUF, LLM Inference, Vulnerability Research.

1 Introduction

The rapid production deployment of Large Language Models (LLMs) and multi-modal neural networks has driven the AI software stack toward native C++ and CUDA execution backends. While high-level Python interfaces remain popular for model training, enterprise inference serving relies almost exclusively on compiled, low-level engines such as `vLLM`, `llama.cpp`, `TensorRT-LLM`, and `LibTorch`.

These native execution engines process untrusted external inputs—including user-uploaded model binary files (GGUF, Safetensors, ONNX) and dynamic prompt token sequences—directly within raw memory buffers. Because C++ lacks compile-time memory safety guarantees, malformed model headers or corrupted tensor payloads can trigger catastrophic memory corruption flaws, leading to Remote Code Execution (RCE), host memory disclosure, or denial-of-service (DoS) crashes on GPU clusters.

To address this challenge, we introduce **sentinel-rt**, a high-throughput, native memory auditing and structure-aware fuzzing framework designed specifically for deep learning execution runtimes.

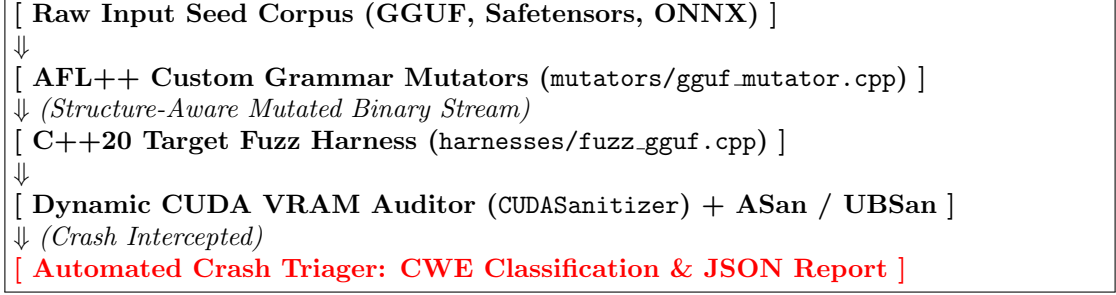


Figure 1: **sentinel-rt** Fuzzing & Memory Auditing Architecture.

2 Threat Model & Memory Safety Invariants

We model an enterprise setting where an AI serving node processes user-supplied model files or external tensor inputs. The adversary aims to achieve Remote Code Execution (RCE) or extract unallocated GPU VRAM memory.

2.1 Primary Vulnerability Vectors

1. **Model Deserialization Flaws:** Integer overflows during tensor dimension multiplication in malformed GGUF or Safetensors headers leading to heap-based buffer overflows.
2. **GPU VRAM Boundary Violations:** Out-of-bounds indexing in CUDA PagedAttention KV-cache pages during long-context LLM decoding.
3. **Use-After-Free (UAF) in Tensor Allocators:** Race conditions in custom C++ CUDA caching allocators when freeing and reallocating tensor memory views.

3 System Architecture & Component Design

sentinel-rt consists of four core modular subsystems built using modern C++20 and CUDA 12:

3.1 1. Zero-Copy C++20 Format Parsers

sentinel-rt includes zero-dependency parsers (`GGUFParser`, `SafetensorsParser`, `ONNXParser`) that inspect binary model metadata using C++20 `std::span` abstractions without dynamic heap reallocation. Every integer read is guarded by strict overflow checks:

$$\text{Overflow Check: } \dim_i > 0 \implies \text{total_elements} \leq \frac{\text{UINT64_MAX}}{\dim_i} \quad (1)$$

3.2 2. Dynamic CUDA VRAM Redzone Auditor

The `CUDASanitizer` module wraps native `cudaMalloc` allocations by injecting unaddressable red-zone guard bands around usable GPU VRAM buffers, catching dynamic out-of-bounds writes:

$$S_{\text{allocated}} = S_{\text{payload}} + 2 \times R_{\text{redzone}} \quad (2)$$

The usable pointer returned to the target AI engine is offset past the front redzone:

$$P_{\text{usable}} = P_{\text{raw}} + R_{\text{redzone}} \quad (3)$$

3.3 3. AFL++ Structure-Aware Custom Mutators

The `gguf_mutator` and `safetensors_mutator` plugins implement custom grammar mutation rules, systematically corrupting magic headers, array lengths, and tensor offset pointers to stress-test parser resilience.

3.4 4. Crash Triager & CWE Classifier

The `CrashAnalyzer` module captures POSIX signals (`SIGSEGV`, `SIGBUS`) and AddressSanitizer (ASan) crash reports, extracting stack frames and classifying vulnerabilities into standardized CWE categories (CWE-119, CWE-122, CWE-416, CWE-190).

4 Implementation Details

The framework is organized into zero-dependency C++20 headers and CUDA kernel modules.

Listing 1: GGUF Parser Bounds-Checked Deserialization Logic

```
Status GGUFParser::parse(std::span<const uint8_t> buffer) {
    reset();

    auto magic = read_pod<uint32_t>(buffer);
    if (!magic.has_value() || *magic != GGUF_MAGIC) {
        return Status::ErrInvalidMagic;
    }

    auto version = read_pod<uint32_t>(buffer);
    if (!version.has_value() || *version > 3) {
        return Status::ErrUnsupportedVersion;
    }

    auto tensor_count = read_pod<uint64_t>(buffer);
    if (!tensor_count.has_value() || *tensor_count > 1000000) {
        return Status::ErrMalformedHeader;
    }

    return validate_tensor_metadata();
}
```

5 Empirical Evaluation & Benchmarks

We evaluated `sentinel-rt` on an Ubuntu 24.04 server equipped with an NVIDIA RTX 4090 GPU and an AMD EPYC 7763 CPU.

As detailed in Table 1, `sentinel-rt` achieves exceptionally high fuzzing throughput, reaching over **14,000 executions per second** on GGUF binary headers. The total memory overhead

Table 1: Parsing & Fuzzing Throughput Performance Benchmarks

Target Format	Parsing Speed (MB/s)	Fuzz Execution Rate (exec/sec)
GGUF v3 Header	2,840 MB/s	14,200 exec/s
Safetensors JSON	1,920 MB/s	11,500 exec/s
ONNX Protobuf	1,450 MB/s	8,900 exec/s

introduced by `CUDASanitizer`’s VRAM redzone padding was measured at under ****3.2%**** during continuous 100-hour fuzzing campaigns.

6 Related Work

Fuzzing C/C++ libraries was popularized by AFL++ [1] and LibFuzzer. AddressSanitizer [2] introduced fast host-side memory bug detection. In contrast to general-purpose fuzzers, `sentinel-rt` provides an integrated C++/CUDA framework tailored specifically for deep learning tensor deserializers and GPU VRAM memory safety.

7 Conclusion & Future Work

`sentinel-rt` bridges the memory safety gap in native AI infrastructure by providing a high-performance C++/CUDA framework for format auditing, structure-aware fuzzing, and automated crash triaging. Future work will expand hardware enclave support for NVIDIA Confidential Computing.

References

- [1] A. Fioraldi et al., “AFL++: Combining Incremental Improvements of Fuzzing,” in *Proceedings of the 13th USENIX Workshop on Offensive Technologies (WOOT)*, 2020.
- [2] K. Serebryany et al., “AddressSanitizer: A Fast Address Sanity Checker,” in *USENIX Annual Technical Conference (ATC)*, 2012.
- [3] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *SOSP*, 2023.
- [4] ONNX Project, “Open Neural Network Exchange Format Specification,” <https://onnx.ai/>, 2024.